escuela
británica de
artes creativas
y tecnología

Buenas prácticas

escuela
británica de
artes creativas
y tecnología

Clases 01–02

Buenas practices

Instrucciones de control: Algebra booleana y condicionales

Confusión entre Asignación y Comparación

Los principiantes a menudo confunden el operador de asignación `=` con el operador de comparación `==`.

Ejemplo:

```python
if x = 10:
    print("x es 10")
```

Corrección:

- Paso 1: Utilice `==` para comparaciones en lugar de `=`.
- Paso 2: El código corregido sería:

```python
if x == 10:
    print("x es 10")
```

escuela
británica de
artes creativas
y tecnología

Clases 01–02

Buenas practices

Instrucciones de control: Algebra booleana y condicionales

Uso Incorrecto de Operadores Lógicos

No comprender las tablas de verdad puede llevar a un uso incorrecto de los operadores lógicos `and`, `or`, y `not`.

Ejemplo:

```python
if edad > 18 or edad < 65:
    print("Edad está en el rango permitido")
```

Esto imprimirá el mensaje para cualquier edad, ya que la condición siempre será verdadera.

Ejemplo:

- Paso 1: Revise las tablas de verdad para los operadores lógicos.
- Paso 2: Corrija la lógica para reflejar la condición deseada:

```python
if edad > 18 and edad < 65:
    print("Edad está en el rango permitido")
```

escuela
británica de
artes creativas
y tecnología

Clases 01–02

Buenas practices

Instrucciones de control: Algebra booleana y condicionales

Indentación Incorrecta en Bloques Condicionales

Python utiliza la indentación para definir bloques de código, y una indentación incorrecta puede causar errores de sintaxis o lógicos.

Ejemplo:

```python
if x > 10:
print("x es mayor que 10")
```

Corrección:

- Paso 1: Asegúrese de que todas las instrucciones dentro de un bloque condicional estén correctamente indentadas.
- Paso 2: El código corregido sería:

```python
if x > 10:
    print("x es mayor que 10")
```

escuela
británica de
artes creativas
y tecnología

Clases 01–02

Buenas practices

Instrucciones de control: Algebra booleana y condicionales

Confusión entre `is` y `==`

Usar `is` en lugar de `==` para comparar valores puede llevar a resultados inesperados, ya que `is` compara identidades de objetos, no valores.

Ejemplo:

```python
if a is b:
    print("a y b son iguales")
```

Esto puede no funcionar como se espera si `a` y `b` son valores iguales pero objetos distintos.

Corrección:

- Paso 1: Utilice `==` para comparar valores y `is` para comparar identidades.
- Paso 2: El código corregido sería:

```python
if a == b:
    print("a y b son iguales")
```

escuela
británica de
artes creativas
y tecnología

¡Mucho éxito en tus estudios!